

SolarBI Pascual: Power BI y Grafana, gobernanza de datos,  
automatización ETL e IoT

Estudiantes:

Andrew Gaitan Giraldo

Juan Diego Naranjo Mora

Asignatura:

Inteligencia de negocios BI

Docente:

Ramiro Grisales Montoya

Institución Universitaria Pascual Bravo

Medellín

2026

## Parte A · Consulta.

### Bloque 1 · Power BI y Grafana

**D1.** Grafana es una herramienta gratuita para vigilar datos que cambian constantemente, como los de un inversor solar. Sus piezas son: **Data sources:** de dónde salen los datos. **Panels:** cada gráfico o medidor. **Dashboards:** la pantalla que reúne los paneles. **Variables:** listas desplegables para cambiar lo que se ve, por ejemplo el inversor. **Alerting:** avisos automáticos si algo falla, como una planta sin producir.

Grafana OSS se instala y mantiene en servidores propios, sin costo de licencia. Grafana Cloud es el servicio ya alojado por Grafana Labs, con plan gratuito limitado y planes pagos. Para SolarBI, OSS sirve para el curso; Cloud ahorra el montaje.

Fuente: Grafana Labs. (s. f.). *Grafana documentation* (secciones: *Data sources, Panels, Dashboards, Variables, Alerting*; *Grafana OSS* y *Grafana Cloud*). <https://grafana.com/docs/>

#### D2.

| <i>Criterio</i>                           | <i>Power BI</i>                                | <i>Grafana</i>                                      |
|-------------------------------------------|------------------------------------------------|-----------------------------------------------------|
| <b><i>Propósito principal</i></b>         | Analizar y explicar el negocio                 | Monitorear sistemas en vivo                         |
| <b><i>Usuario típico</i></b>              | Analista, gerente                              | Ingeniero de operación, técnico                     |
| <b><i>Conexión a los datos</i></b>        | Archivos, bases, servicios en la nube          | Bases de series de tiempo y SQL                     |
| <b><i>Modelo semántico y lenguaje</i></b> | Modelo propio con relaciones; DAX              | Sin modelo propio; usa la consulta del origen (SQL) |
| <b><i>Actualización y tiempo real</i></b> | Programada (o DirectQuery)                     | Auto-refresh cada pocos segundos                    |
| <b><i>Alertas</i></b>                     | Básicas, sobre indicadores                     | Muy completas, con envío a correo o chat            |
| <b><i>Licenciamiento y costo</i></b>      | Pro/Premium por usuario o capacidad            | OSS gratis; Cloud con plan gratuito y pagos         |
| <b><i>Control de acceso</i></b>           | Roles, seguridad por fila, espacios de trabajo | Usuarios, equipos y permisos por carpeta            |
| <b><i>Facilidad de uso</i></b>            | Arrastrar y soltar                             | Más técnico, requiere consultas                     |

#### D3. Tipos de dashboard y KPIs

Los tipos de KPIs son: **Operativo:** muestra lo que pasa ahora y exige reacción rápida. **Analítico:** permite explorar el porqué de los resultados. **Estratégico:** resume pocos indicadores para decidir a largo plazo.

| <i>KPI</i>                       | <i>Tipo</i> | <i>Herramienta</i> | <i>Por qué</i>                        |
|----------------------------------|-------------|--------------------|---------------------------------------|
| <i>Energía generada</i>          | Analítico   | Power BI           | Se compara por día, mes y planta      |
| <i>Yield y Performance Ratio</i> | Analítico   | Power BI           | Requiere comparar y explicar causas   |
| <i>Ahorro</i>                    | Estratégico | Power BI           | Interesa a directivos, es acumulado   |
| <i>Disponibilidad y alarmas</i>  | Operativo   | Grafana            | Necesita tiempo casi real y avisos    |
| <i>% de datos válidos</i>        | Operativo   | Grafana            | Detecta fallas de captura al instante |

#### **D4. Almacenamiento en Power BI**

*Import* copia los datos dentro de Power BI: es rápido, pero se ve la foto del último refresco.

*DirectQuery* consulta la fuente cada vez: está más al día, pero es más lento y exige una fuente potente. El refresco programado repite la copia en horarios fijos. En SolarBI usaríamos *Import* con refresco programado: el análisis no necesita segundos de precisión y los gráficos se cargan ágilmente. Grafana, en cambio, se auto-actualiza cada pocos segundos consultando la fuente en vivo; por eso lo dejamos para lo operativo.

Fuente: Microsoft. (s. f.). *Power BI documentation* (secciones: Storage mode, Data refresh, Row-level security, Slicers). Microsoft Learn. <https://learn.microsoft.com/power-bi/>

#### **D5. Interactividad**

Las variables de Grafana son listas desplegables que se insertan dentro de la consulta: al cambiarlas, el panel pide otros datos a la base. Los segmentadores y filtros de Power BI actúan sobre datos ya cargados y filtran lo que se muestra. Variable propuesta: dispositivo, con la lista de inversores. La consulta pasa de traer todos los equipos a esto:

```
SELECT hora, energia_kwh
```

```
FROM lectura_inversor
```

```
WHERE dispositivo_id = '$dispositivo'
```

Al elegir "Inversor 3", Grafana sustituye el dispositivo y el panel muestra solo ese equipo.

### **Bloque 2 · Gobernanza de datos**

#### **G1. Data governance**

La gobernanza de datos son las reglas y responsables que dicen quién decide sobre los datos y cómo se usan. *data management* es el trabajo diario de guardarlos, moverlos y mantenerlos. *data quality* mide si son correctos y completos. Según DAMA-DMBOK, la gobernanza está en el centro y orienta a las demás áreas. En SolarBI, la gobernanza define que la energía se mide en kWh y quién aprueba cambios; la gestión es el ETL que la carga; la calidad es el porcentaje de datos válidos.

#### **G2. Roles**

| <i>Rol de gobernanza</i> | <i>Qué hace</i>                            | <i>Rol en SolarBI</i>     |
|--------------------------|--------------------------------------------|---------------------------|
| <i>Data owner</i>        | Responde por un dato y decide quién lo usa | Product owner             |
| <i>Data steward</i>      | Vigila su calidad y su significado         | BI analyst y data modeler |
| <i>Data custodian</i>    | Lo guarda y protege técnicamente           | Data engineer             |

El *owner* decide, el *steward* revisa que esté bien definido y el *custodian* lo mantiene seguro y disponible.

### G3. Contrato de datos

Es un acuerdo escrito entre quien entrega los datos y quien los usa. Debe incluir: un esquema; nombres y tipos de campos. Unidades: por ejemplo, kWh. Frecuencia: cada cuánto llegan. Responsable: a quién llamar si falla. Niveles de calidad: por ejemplo, mínimo 95 % de datos válidos.

Si la fuente cambia un campo sin avisar, el cuadro de mando puede mostrar errores, vacíos o cifras equivocadas sin que nadie lo note. Por ejemplo, si la energía pasa de kWh a Wh, la producción se vería mil veces mayor.

### G4. Fuente única y catálogo de KPIs

La fuente única de verdad es un solo lugar oficial donde se calcula cada cifra. El catálogo de KPIs es la lista que define cada indicador. Fichas:

| <i>Campo</i>       | <i>Energía generada</i>        | <i>Performance Ratio</i>        |
|--------------------|--------------------------------|---------------------------------|
| <i>Fórmula</i>     | Suma de la energía por periodo | Energía real ÷ energía esperada |
| <i>Unidad</i>      | kWh                            | %                               |
| <i>Fuente</i>      | Tabla de hechos de lecturas    | Lecturas + irradiancia          |
| <i>Responsable</i> | Product owner                  | BI analyst                      |

|                   |        |        |
|-------------------|--------|--------|
| <b>Frecuencia</b> | Diaria | Diaria |
|-------------------|--------|--------|

Si un KPI se calcula en DAX y en SQL por separado, pueden dar resultados distintos y la gente deja de confiar en el tablero.

### G5. Ley 1581 de 2012

Esta ley protege los datos que identifican a una persona. Los datos de SolarBI describen equipos, no personas, así que en principio no son personales. Si el tablero mostrará qué operador atendió cada alarma, sí serían personales: habría que informar al operador, tener su autorización y limitar quién lo ve. Nuestra postura: usar solo lo necesario, preferir códigos anónimos, controlar accesos y no publicar datos de personas sin motivo claro.

Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales.*

[http://www.secretariassenado.gov.co/senado/basedoc/ley\\_1581\\_2012.html](http://www.secretariassenado.gov.co/senado/basedoc/ley_1581_2012.html)

### Bloque 3 · Automatización de procesos ETL

**E1.** ETL y ELT se diferencian en cuándo se transforman. En ETL los datos se arreglan antes de guardarse; en ELT se guardan primero como llegan y se arreglan después, ya dentro de la base. SolarBI se parece más a ELT. Bronze guarda las lecturas del inversor sin modificar. En Silver se corrigen unidades, se revisan huecos y se calcula qué porcentaje de datos es válido. Gold deja los resúmenes, como fact\_energia\_dia. Si hay un error en Silver, se vuelve a generar desde Bronze. Power Query sí es ETL, pero sencillo: extrae, transforma y carga al modelo, aunque no agenda tareas ni guarda registro de lo que hizo. Lo dejamos para retoques finales.

Referencias: Microsoft. (s. f.). *What is Power Query? y Incremental refresh for semantic models* (RangeStart y RangeEnd). Microsoft Learn. <https://learn.microsoft.com/power-query/>

**E2.** En la carga completa se borra todo y se vuelve a cargar desde cero; es simple, pero con el tiempo se vuelve pesada porque el histórico crece. En la incremental solo se cargan los datos nuevos o modificados desde la última vez. Idempotente quiere decir que correr la carga varias veces con los mismos datos siempre deja el mismo resultado. En fact\_energia\_dia lo logramos con la clave (fecha\_key, dispositivo\_key) y un UPSERT: si esa combinación de día y dispositivo ya está, se actualiza; si no, se agrega. Por eso, si toca repetir un día porque llegaron lecturas tarde, no aparecen filas repetidas.

```
INSERT INTO fact_energia_dia (fecha_key, dispositivo_key, energia_kwh,
pct_datos_validos)
SELECT fecha_key, dispositivo_key, SUM(energia_kwh),
      AVG(CAST(es_valido AS INT)) * 100
FROM silver_telemetria
WHERE fecha_key >= :ultima_fecha_cargada
GROUP BY fecha_key, dispositivo_key
ON CONFLICT (fecha_key, dispositivo_key)
DO UPDATE SET energia_kwh = EXCLUDED.energia_kwh,
      pct_datos_validos = EXCLUDED.pct_datos_validos;
```

### E3. Tabla comparativa de tres formas de automatizar un flujo.

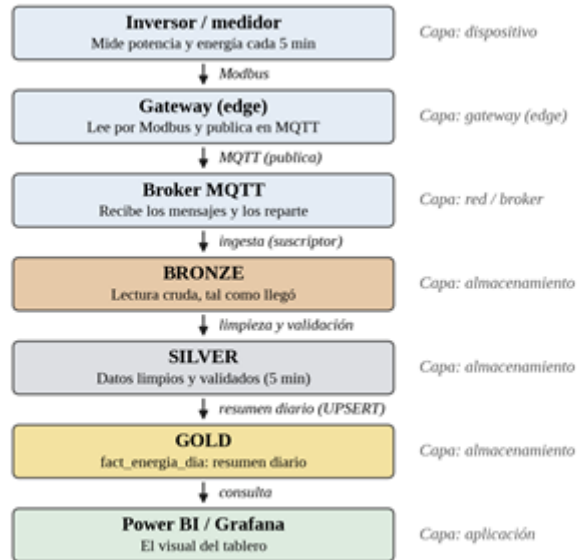
|                                        | <b>cron / Programador de tareas</b>                               | <b>Apache Airflow</b>                                                                                   | <b>Actualización programada de Power BI</b>                                                             |
|----------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>¿Qué es?</b>                        | El despertador del computador: lanza un programa a una hora fija. | Un director de orquesta: organiza, programa y vigila flujos de varias tareas (DAGs escritos en Python). | Una función del servicio de Power BI que vuelve a cargar el modelo desde sus fuentes a horas definidas. |
| <b>¿Qué automatiza?</b>                | Un script suelto (por ejemplo, el que carga Bronze).              | Toda la cadena: primero Bronze, luego Silver, luego Gold.                                               | Solo la recarga del modelo del tablero.                                                                 |
| <b>¿Respeto el orden entre tareas?</b> | No.                                                               | Sí, mediante dependencias.                                                                              | No; solo refresca el modelo.                                                                            |
| <b>Si algo falla</b>                   | Solo lo que se programe en el propio script.                      | Reintentos, alertas e historial en una interfaz web.                                                    | Muestra el historial y puede avisar del fallo.                                                          |
| <b>¿Cuándo conviene?</b>               | Tareas simples, prototipos y equipos pequeños.                    | Procesos con varios pasos que dependen unos de otros y necesitan trazabilidad.                          | Cuando el producto final es un tablero de Power BI con datos ya listos en Gold.                         |
| <b>Limitación</b>                      | Sin visibilidad ni control de dependencias.                       | Más difícil de montar y mantener.                                                                       | No prepara los datos; solo actualiza lo que ya existe.                                                  |

Las tres sirven para que el proceso corra solo, pero a distinto nivel. Airflow es el único que entiende el orden de las tareas (Bronze, luego Silver, luego Gold), reintentará si algo falla y muestra todo en una pantalla. cron es un simple despertador, y la actualización de Power BI solo recarga el tablero, sin preparar los datos que llegan antes. Para SolarBI, cron alcanza para un prototipo; Airflow sería lo ideal para controlar toda la cadena; y la actualización de Power BI va al final, cuando Gold ya está listo.

**E4.** Con la actualización incremental Power BI no recarga toda la tabla cada vez, solo la parte reciente. Para configurarla se crean dos parámetros de fecha, RangeStart y RangeEnd, y se usan para filtrar la columna de fecha de la tabla. Después se elige la política: cuánto histórico se conserva y cuántos periodos recientes se refrescan. Al publicar en el servicio, Power BI cambia esos valores por su cuenta y solo actualiza las partes recientes. Sirve sobre todo con tablas muy grandes. En SolarBI, fact\_energia\_dia tiene una fila por dispositivo y día, entonces es pequeña y se puede recargar completa. Cambiaría si cargáramos las lecturas de cada 5 minutos.

### **Bloque 4 · Internet de las Cosas (IoT)**

**II.** Una lectura pasa por cinco capas. El dispositivo mide potencia, energía y voltaje. El gateway está cerca de la planta: lee los datos por Modbus, los publica en MQTT y puede guardarlos temporalmente si se cae el internet. El broker es el cartero que recibe los mensajes y los reparte. En el almacenamiento, la lectura llega cruda a Bronze, se limpia en Silver y se resume por día en Gold. Por último, la aplicación Power BI y Grafana lee esos datos y los convierte en el visual del tablero.



**I2.** MQTT funciona como un sistema de canales. Los dispositivos no se hablan entre sí: quien mide publica un mensaje en un topic (una especie de dirección con niveles separados por "/") y el broker lo entrega a todos los que se suscribieron a ese topic. Los comodines + y # permiten suscribirse a varios a la vez. El QoS indica cuánta garantía de entrega se quiere: 0 envía sin confirmar y puede perder mensajes; 1 asegura que llegue al menos una vez, aunque podría repetirse; 2 asegura que llegue exactamente una vez, pero es más lento. Para SolarBI usamos QoS 1: un duplicado se absorbe con el UPSERT.

```
pascualbravo/solar/<sitio>/<dispositivo>/<variable>
```

Ejemplos (los nombres de sitio son ilustrativos):

```
pascualbravo/solar/bloque-m/inv01/potencia_ac_w
pascualbravo/solar/bloque-m/inv01/energia_dia_kwh
pascualbravo/solar/bloque-m/med01/energia_kwh
```

Suscripciones con comodines:

```
pascualbravo/solar/bloque-m/#
pascualbravo/solar/+inv01/#
```

Referencias: OASIS. (2019). *MQTT Version 5.0*.

<https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

**I3.** Comparación de Modbus, MQTT y OPC UA.

|                          | Modbus (RTU/TCP)                                                                                    | MQTT                                                                             | OPC UA                                                              |
|--------------------------|-----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Cómo se comunican</b> | Un maestro pregunta y el equipo responde (sondeo). RTU va por cable serie (RS-485); TCP va por red. | Publicación y suscripción mediante un broker; nadie le habla directo a nadie.    | Cliente-servidor, y también publicación-suscripción.                |
| <b>Uso típico</b>        | Leer valores de equipos industriales: inversores, medidores, PLC.                                   | Enviar telemetría de muchos dispositivos al servidor o la nube con poco consumo. | Integrar equipos con sistemas SCADA o MES con datos bien descritos. |

|                                     |                                                                                              |                                                                       |                                                                                |
|-------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>Formato de los datos</b>         | Registros numéricos de 16 bits; hay que saber qué significa cada número (mapa de registros). | Contenido libre; lo más común es JSON.                                | Objetos con nombre, tipo y unidades (modelo de información).                   |
| <b>Seguridad</b>                    | Casi ninguna propia; depende de proteger la red.                                             | Cifrado TLS, usuario y contraseña, y permisos por topic en el broker. | Certificados digitales, cifrado, firma, autenticación de usuarios y auditoría. |
| <b>¿Lo esperaríamos en SolarBI?</b> | Sí, en inversores y medidores (a menudo con modelos SunSpec).                                | Sí, en el gateway hacia el servidor.                                  | Quizá en plantas grandes con SCADA; menos probable en una planta pequeña.      |

En una planta solar lo más probable es encontrar Modbus en los inversores y medidores, porque es el protocolo más común y sencillo en ese tipo de equipos. MQTT lo ubicaríamos en el gateway, para mandar los datos hacia el servidor. OPC UA aparece más en plantas grandes con SCADA: es más completo y seguro, pero también más complejo. La diferencia clave es la seguridad: Modbus casi no trae, así que se protege la red; OPC UA usa certificados y cifrado desde el diseño.

Referencias: OPC Foundation. (s. f.). *OPC Unified Architecture*.  
<https://opcfoundation.org/about/opc-technologies/opc-ua/>

**I4.** Batch es procesar los datos acumulados de una sola vez, por ejemplo cada noche; streaming es procesarlos uno por uno apenas llegan. En SolarBI llega una lectura cada 5 minutos (288 al día por variable) y `fact_energia_dia` las junta en una sola fila por día. Ganamos tablas más livianas, consultas rápidas y KPIs fáciles (kWh por día, yield, PR). Perdemos el detalle dentro del día: la curva de potencia, las caídas por nubes o la hora exacta de una falla. Por eso Silver se queda con los datos de 5 minutos. Power BI sirve mejor para lo diario e histórico; Grafana, para lo de 5 minutos y las alertas casi en vivo.

### ***Bloque 5 · Control de versiones***

Git es el programa que registra el historial de cambios de los archivos; GitHub es la página web donde se guarda ese historial y el equipo colabora. En SolarBI, el *repository* es la carpeta del proyecto; un *commit* es una foto guardada con mensaje; una *branch* es una copia paralela para probar sin dañar lo principal; y un *pull request* es la petición de revisar y unir esos cambios. El `.pbix` es un archivo cerrado: Git no muestra qué cambió adentro ni permite unir el trabajo de dos personas. El `.pbip` lo separa en archivos de texto legibles, que sí se pueden comparar. Un dashboard de Grafana se exporta como JSON y se guarda en el repositorio.

## ***Parte B · Evidencias***

### ***Parte 1***

```

simulador1.py > ...
1  import csv
2  import math
3  import os
4  import random
5  from datetime import datetime, timedelta
6  import pandas as pd
7
8  inicio = datetime(2026, 10, 5, 0, 0)
9  os.makedirs("data/bronze", exist_ok=True)
10 ruta_bronze = "data/bronze/telemetry.csv"
11
12 with open(ruta_bronze, "w", newline="", encoding="utf-8") as f:
13     w = csv.writer(f)
14     w.writerow(["ts", "dispositivo_id", "p_ac_kw", "irradiancia_wm2", "temp_modulo_c"])
15
16     for i in range(3 * 288):
17         ts = inicio + timedelta(minutes=5 * i)
18         sol = max(0.0, math.sin(math.pi * (ts.hour + ts.minute / 60 - 6) / 12))
19         irr = round(1000 * sol * random.uniform(0.7, 1.0), 1)
20         p_ac = round(5.0 * irr / 1000 * random.uniform(0.80, 0.90), 3)
21         fila = [ts.isoformat(sep=" "), 1, p_ac, irr, round(22 + 30 * sol, 1)]
22
23         r = random.random()
24         if r < 0.02:
25             fila[2] = -1
26         elif r < 0.04:
27             fila[3] = ""
28
29         w.writerow(fila)
30
31         if r > 0.98:
32             w.writerow(fila)
33 print("✓ Paso 1 completado: Capa Bronze creada en 'data/bronze/telemetry.csv'\n")

```

Se generó una simulación de 3 días de telemetría de un inversor solar en intervalos de 5 minutos guardados en data/bronze/telemetry.csv.

## Parte 2

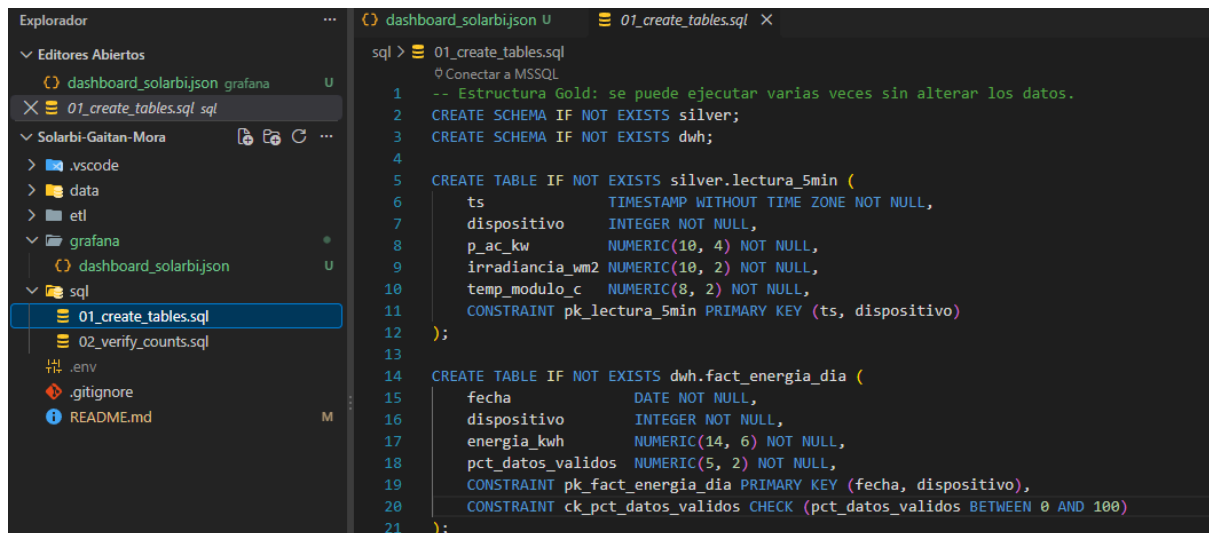
```

simulador.py X
eti > simulador.py > ...
1  import os
2  import pandas as pd
3
4  # Límites físicos (ajusten la potencia a la capacidad real de la planta)
5  P_MAX_KW = 6.0
6  IRR_MAX = 1500
7  TEMP_MIN, TEMP_MAX = -10, 80
8
9  ruta_bronze = "data/bronze/telemetria.csv"
10 ruta_silver_dir = "data/silver"
11 ruta_silver_clean = os.path.join(ruta_silver_dir, "telemetria_clean.csv")
12 os.makedirs(ruta_silver_dir, exist_ok=True)
13
14 df = pd.read_csv(ruta_bronze)
15 filas_leidas = len(df)
16
17 for col in ["p_ac_kw", "irradiancia_wm2", "temp_modulo_c"]:
18     df[col] = pd.to_numeric(df[col], errors="coerce")
19
20 # Regla 1: datos faltantes
21 mask_faltantes = df.isnull().any(axis=1)
22 rechazadas_faltantes = int(mask_faltantes.sum())
23 resto = df[~mask_faltantes]
24
25 # Regla 2: duplicados (solo entre filas completas)
26 mask_dup = resto.duplicated(subset=["ts", "dispositivo_id"], keep="first")
27 rechazadas_duplicados = int(mask_dup.sum())
28 resto = resto[~mask_dup]
29
30 # Regla 3: rango físico válido (solo entre las que quedan)
31 mask_rango = (
32     (resto["p_ac_kw"] < 0) | (resto["p_ac_kw"] > P_MAX_KW) |
33     (resto["irradiancia_wm2"] < 0) | (resto["irradiancia_wm2"] > IRR_MAX) |
34     (resto["temp_modulo_c"] < TEMP_MIN) | (resto["temp_modulo_c"] > TEMP_MAX)
35 )
36 rechazadas_rango = int(mask_rango.sum())
37 df_silver = resto[~mask_rango].copy()
38
39 filas_validas = len(df_silver)
40 filas_rechazadas_total = filas_leidas - filas_validas
41 pct_validas = (filas_validas / filas_leidas) * 100 if filas_leidas else 0.0
42
43 # Verificación: la suma por regla debe coincidir con el total
44 assert rechazadas_faltantes + rechazadas_duplicados + \
45     rechazadas_rango == filas_rechazadas_total
46
47 df_silver.to_csv(ruta_silver_clean, index=False, encoding="utf-8")
48
49 print("=" * 50)
50 print("      REPORTE DE CALIDAD DE DATOS (SILVER)")
51 print("=" * 50)
52 print(f"Filas leídas (Bronze):           {filas_leidas}")
53 print(f"Rechazadas por datos faltantes:    {rechazadas_faltantes}")
54 print(f"Rechazadas por duplicados:         {rechazadas_duplicados}")
55 print(f"Rechazadas por rango inválido:     {rechazadas_rango}")

```

Se procesó el dataset crudo mediante Pandas aplicando tres reglas estrictas de calidad para eliminar registros nulos, duplicados y lecturas fuera de rango físico.

### Parte 3

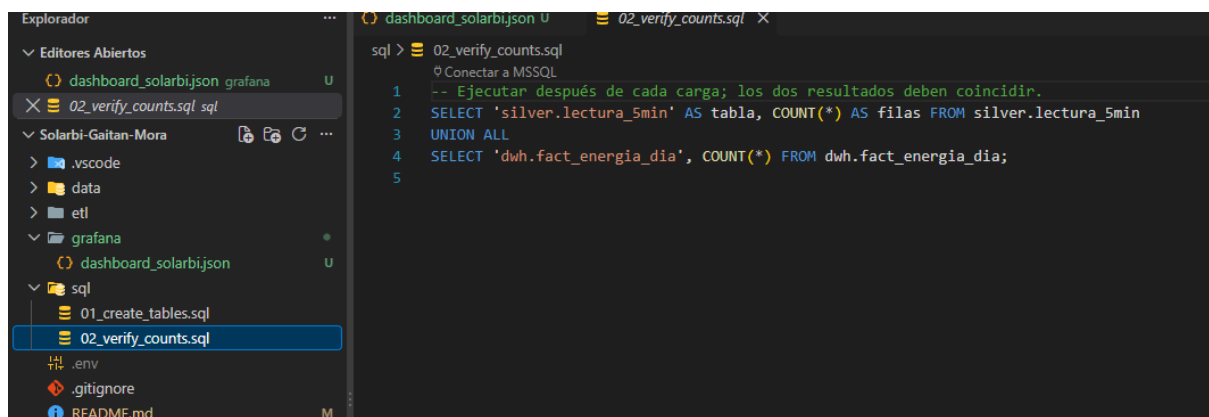


The screenshot shows the VS Code interface. On the left, the Explorer sidebar displays the file structure of the 'Solarbi-Gaitan-Mora' project, with '01\_create\_tables.sql' selected under the 'sql' folder. The main editor window shows the content of this file, which contains SQL code to create schemas and tables in a PostgreSQL database.

```
sql > 01_create_tables.sql
-- Conectar a MSSQL

1  -- Estructura Gold: se puede ejecutar varias veces sin alterar los datos.
2  CREATE SCHEMA IF NOT EXISTS silver;
3  CREATE SCHEMA IF NOT EXISTS dwh;
4
5  CREATE TABLE IF NOT EXISTS silver.lectura_5min (
6      ts                TIMESTAMP WITHOUT TIME ZONE NOT NULL,
7      dispositivo        INTEGER NOT NULL,
8      p_ac_kw            NUMERIC(10, 4) NOT NULL,
9      irradiancia_wm2    NUMERIC(10, 2) NOT NULL,
10     temp_modulo_c      NUMERIC(8, 2) NOT NULL,
11     CONSTRAINT pk_lectura_5min PRIMARY KEY (ts, dispositivo)
12 );
13
14 CREATE TABLE IF NOT EXISTS dwh.fact_energia_dia (
15     fecha              DATE NOT NULL,
16     dispositivo        INTEGER NOT NULL,
17     energia_kwh         NUMERIC(14, 6) NOT NULL,
18     pct_datos_validos   NUMERIC(5, 2) NOT NULL,
19     CONSTRAINT pk_fact_energia_dia PRIMARY KEY (fecha, dispositivo),
20     CONSTRAINT ck_pct_datos_validos CHECK (pct_datos_validos BETWEEN 0 AND 100)
21 );
```

Se definieron los esquemas silver y dwh y sus tablas: lecturas cada cinco minutos y resumen diario de energía. Las claves primarias evitar duplicados.

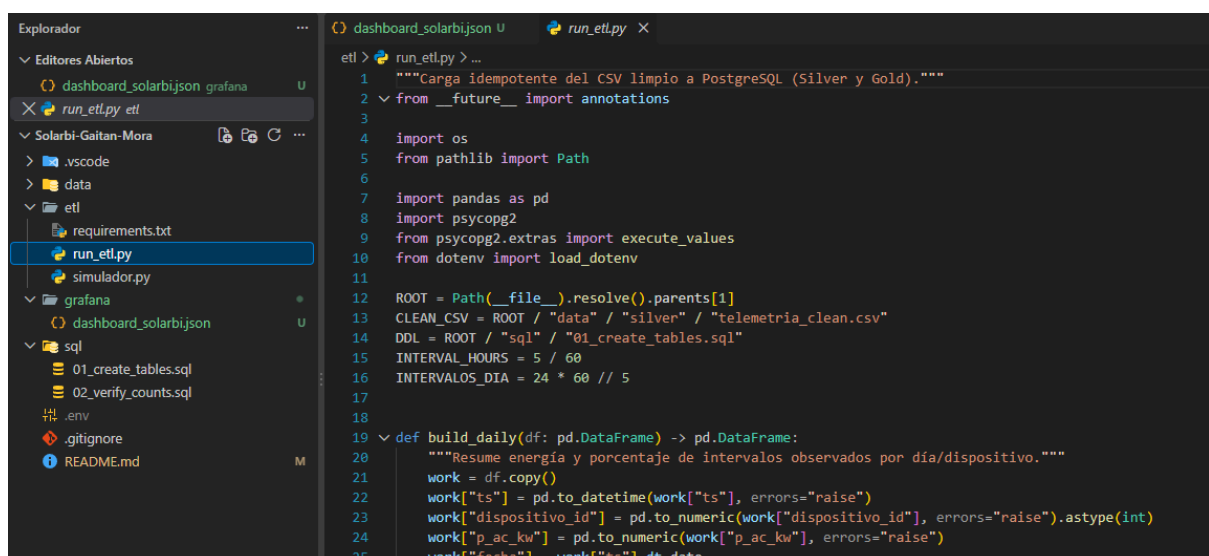


The screenshot shows the VS Code interface. The Explorer sidebar shows '02\_verify\_counts.sql' selected under the 'sql' folder. The main editor window displays the SQL code for a query that verifies the counts of records in the 'silver.lectura\_5min' and 'dwh.fact\_energia\_dia' tables.

```
sql > 02_verify_counts.sql
-- Conectar a MSSQL

1  -- Ejecutar después de cada carga; los dos resultados deben coincidir.
2  SELECT 'silver.lectura_5min' AS tabla, COUNT(*) AS filas FROM silver.lectura_5min
3  UNION ALL
4  SELECT 'dwh.fact_energia_dia', COUNT(*) FROM dwh.fact_energia_dia;
5
```

Se preparó una consulta para contar los registros de ambas tablas. Al ejecutarla después de cada carga, se comprueba que los conteos se mantengan.



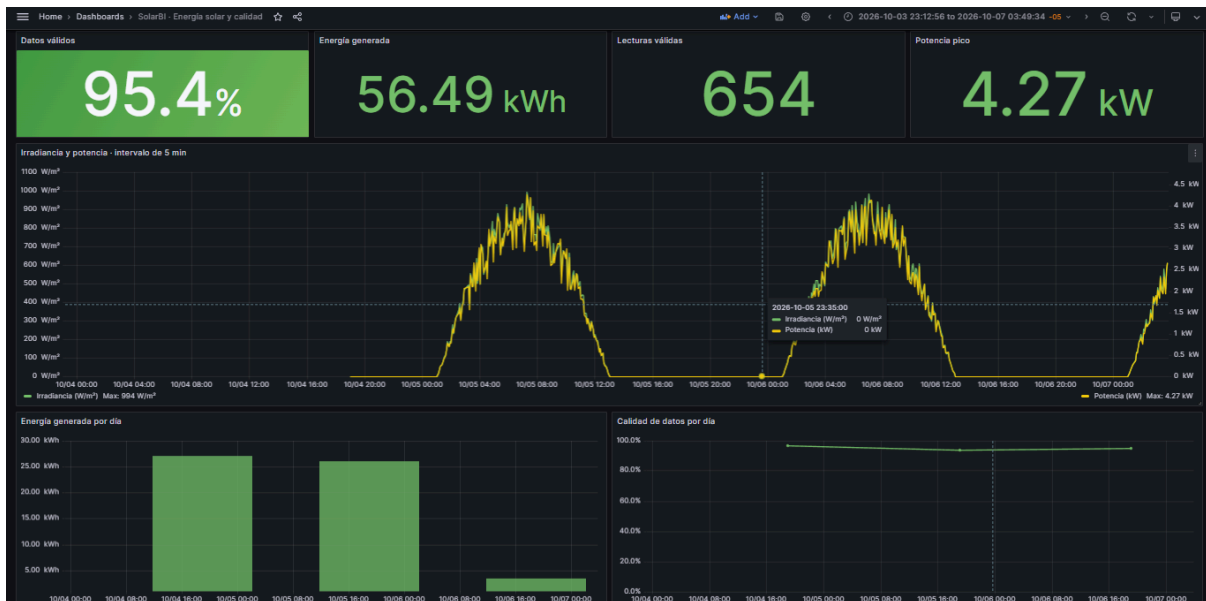
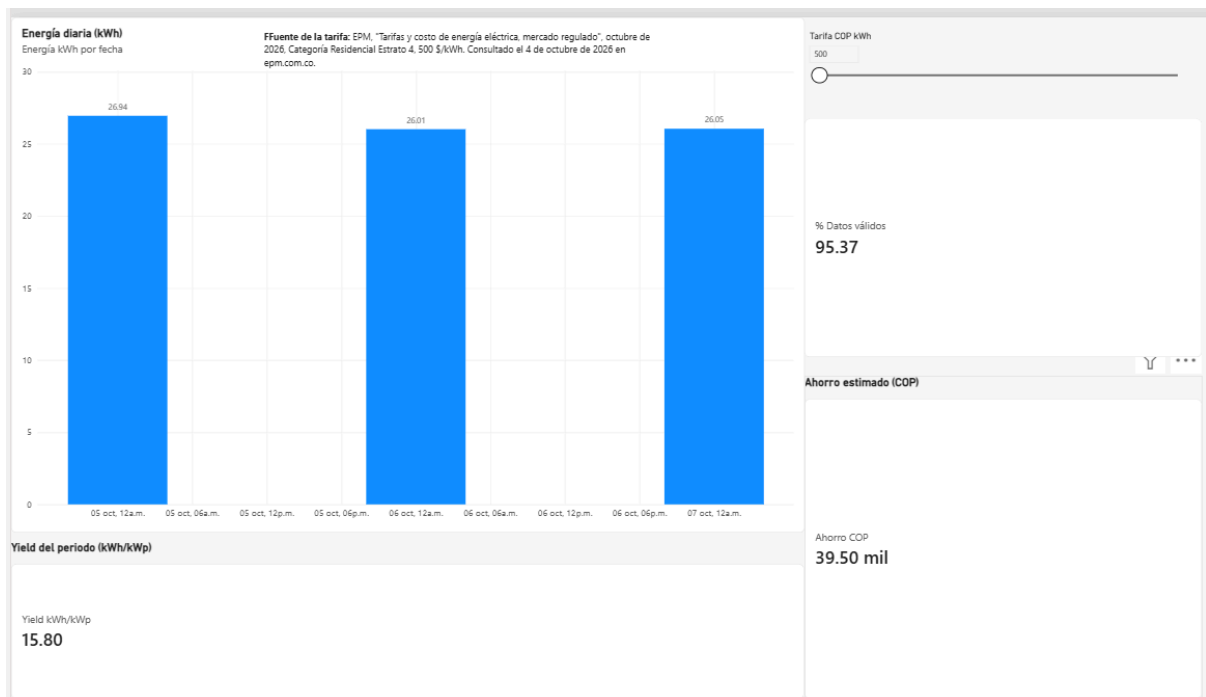
The screenshot shows the VS Code interface. The Explorer sidebar shows 'run\_etl.py' selected under the 'etl' folder. The main editor window displays the Python code for the ETL process, which includes imports, file path definitions, and a function to build a daily data frame from CSV files.

```
etl > run_etl.py > ...

1  """Carga idempotente del CSV limpio a PostgreSQL (Silver y Gold)."""
2  from __future__ import annotations
3
4  import os
5  from pathlib import Path
6
7  import pandas as pd
8  import psycopg2
9  from psycopg2.extras import execute_values
10 from dotenv import load_dotenv
11
12 ROOT = Path(__file__).resolve().parents[1]
13 CLEAN_CSV = ROOT / "data" / "silver" / "telemetria_clean.csv"
14 DDL = ROOT / "sql" / "01_create_tables.sql"
15 INTERVAL_HOURS = 5 / 60
16 INTERVALOS_DIA = 24 * 60 // 5
17
18
19 def build_daily(df: pd.DataFrame) -> pd.DataFrame:
20     """Resume energía y porcentaje de intervalos observados por día/dispositivo."""
21     work = df.copy()
22     work["ts"] = pd.to_datetime(work["ts"], errors="raise")
23     work["dispositivo_id"] = pd.to_numeric(work["dispositivo_id"], errors="raise").astype(int)
24     work["p_ac_kw"] = pd.to_numeric(work["p_ac_kw"], errors="raise")
25     work["fecha"] = work["ts"].dt.date
```

Se implementó etl/run\_etl.py como punto único de entrada. El script lee las lecturas limpias, calcula energía y porcentaje válido, y carga los resultados en PostgreSQL de forma idempotente.

## Paso 5. Evidencias del dashboard en Grafana y en PowerBI



## Paso 6 · Reflexión

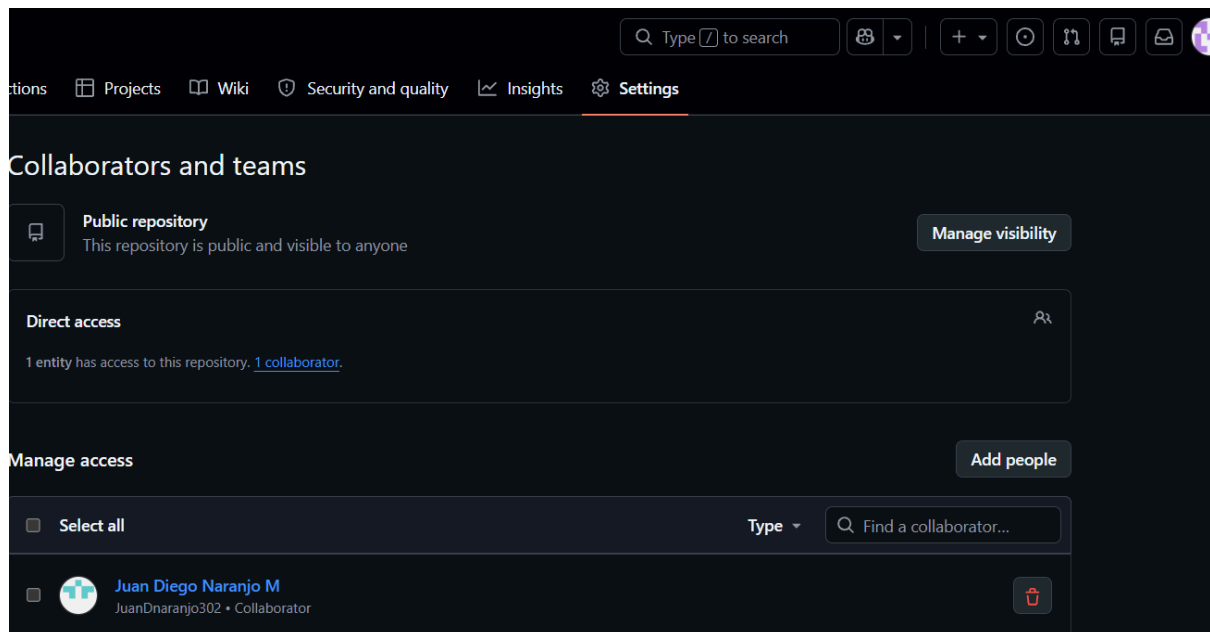
Al comparar Grafana y Power BI con los mismos datos, entendimos que cada herramienta responde a necesidades diferentes. Grafana nos permitió seguir la potencia y la irradiancia en intervalos de tiempo y observar rápidamente sus cambios; sin embargo, conectar PostgreSQL con Supabase, ajustar el rango de fechas y organizar los paneles requirió varios intentos. Power BI nos pareció más cómodo para resumir la información y presentarla en informes, aunque también tuvimos que conectar y preparar los datos antes de crear las visualizaciones.

Por eso, usaríamos Grafana para el operador de la planta, que necesita vigilar el comportamiento de las lecturas y detectar cambios durante la operación. Para un directivo elegimos Power BI, porque presenta los indicadores de forma resumida y facilita comparar resultados para tomar decisiones

### Parte C.

### Evidencias adicionales

### Colaboradores de GitHub



### Commits



Link del repositorio: <https://github.com/Andw923/Solarbi-Gaitan-Giraldo-Mora-Naranjo.git>

### Referencias:

- Grafana Labs. (s. f.). Grafana documentation (secciones: Data sources, Panels, Dashboards, Variables, Alerting; Grafana OSS y Grafana Cloud). <https://grafana.com/docs/>

- Microsoft. (s. f.). *Power BI documentation* (secciones: *Storage mode*, *Data refresh*, *Row-level security*, *Slicers*). Microsoft Learn. <https://learn.microsoft.com/power-bi/>
- Congreso de la República de Colombia. (2012). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. [http://www.secretariassenado.gov.co/senado/basedoc/ley\\_1581\\_2012.html](http://www.secretariassenado.gov.co/senado/basedoc/ley_1581_2012.html)
- Microsoft. (s. f.). *What is Power Query? y Incremental refresh for semantic models* (*RangeStart* y *RangeEnd*). Microsoft Learn. <https://learn.microsoft.com/power-query/>
- OPC Foundation. (s. f.). *OPC Unified Architecture*. <https://opcfoundation.org/about/opc-technologies/opc-ua/>
- OASIS. (2019). *MQTT Version 5.0*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>